

```

import yfinance as yf
import pandas as pd
import requests
import json
import os
from datetime import datetime
import pytz

# -----
# 環境変数
# -----
ACCESS_TOKEN = os.getenv('LINE_ACCESS_TOKEN')
USER_ID = os.getenv('LINE_USER_ID')

# -----
# LINE送信
# -----
def send_line(text: str) -> None:
    if not text:
        return
    url = "https://api.line.me/v2/bot/message/push"
    headers = {
        "Content-Type": "application/json",
        "Authorization": f"Bearer {ACCESS_TOKEN}",
    }
    data = {"to": USER_ID, "messages": [{"type": "text", "text": text}]}
    try:
        res = requests.post(url, headers=headers, data=json.dumps(data), timeout=
        res.raise_for_status()
    except Exception as e:
        print(f"LINE送信エラー: {e}")

# -----
# 市場安全チェック
# -----
def is_market_safe() -> tuple[bool, str]:
    jst = datetime.now(pytz.timezone('Asia/Tokyo'))
    weekday = jst.weekday()

    # 土日は休止
    if weekday >= 5:
        return False, "土日休止"

```

```

# 月曜早朝リスク（6～9時のみ）
if weekday == 0 and 6 <= jst.hour < 9:
    return False, "月曜早朝リスク"

# 早朝メンテ（月曜以外の6時台・7時30分未満）
# * 月曜は上の条件で既にブロック済みなので重複しない
if weekday != 0 and (jst.hour == 6 or (jst.hour == 7 and jst.minute < 30)):
    return False, "早朝メンテ"

return True, "取引可能"

# -----
# ATR計算（True Range の14期間移動平均）
# -----
def calc_atr(df: pd.DataFrame, period: int = 14) -> float:
    high = df['High']
    low = df['Low']
    prev_close = df['Close'].shift(1)

    tr = pd.concat([
        high - low,
        (high - prev_close).abs(),
        (low - prev_close).abs(),
    ], axis=1).max(axis=1)

    atr_series = tr.rolling(period).mean()
    val = atr_series.iloc[-1]
    if pd.isna(val):
        raise ValueError("ATR計算結果がNaN（データ不足の可能性）")
    return float(val)

# -----
# ADX計算（修正版：事前に変数化してインデックスズレを回避）
# -----
def calc_adx(df: pd.DataFrame, period: int = 14) -> tuple[float, float, float]:
    """
    Returns: (adx, plus_di_last, minus_di_last)
    """
    high_diff = df['High'].diff()
    low_diff = df['Low'].diff()
    minus_move = (-low_diff)

    # 正方向：前足より上昇した幅
    # 下落幅を正值に変換

    # +DM: 上昇幅が下落幅より大きく、かつ正值のとき
    plus_dm = high_diff.where((high_diff > 0) & (high_diff > minus_move), 0.0)

```

```

# -DM: 下落幅が上昇幅より大きく、かつ正值のとき
minus_dm = minus_move.where((minus_move > 0) & (minus_move > high_diff), 0.0)

# True Range
prev_close = df['Close'].shift(1)
tr = pd.concat([
    df['High'] - df['Low'],
    (df['High'] - prev_close).abs(),
    (df['Low'] - prev_close).abs(),
], axis=1).max(axis=1)

atr_adx = tr.rolling(period).mean()
plus_di = 100 * (plus_dm.rolling(period).mean() / atr_adx)
minus_di = 100 * (minus_dm.rolling(period).mean() / atr_adx)

dx = (abs(plus_di - minus_di) / (plus_di + minus_di).replace(0, 1e-9)) * 100
adx = dx.rolling(period).mean()

return float(adx.iloc[-1]), float(plus_di.iloc[-1]), float(minus_di.iloc[-1])

# -----
# RSI計算
# -----
def calc_rsi(series: pd.Series, period: int = 14) -> float:
    delta = series.diff()
    gain = delta.where(delta > 0, 0.0).rolling(period).mean()
    loss = (-delta.where(delta < 0, 0.0)).rolling(period).mean()
    rs = gain / loss.replace(0, 1e-9)
    rsi = 100 - (100 / (1 + rs))
    return float(rsi.iloc[-1])

# -----
# SL幅の動的計算 (ATR × 市場状態を加味した乗数)
# -----
def dynamic_sl_multiplier(rsi: float, atr: float, atr_mean: float) -> float:
    """
    ATRが平均より大きい（ボラ高）ならSLを広げ、
    RSIが極値ならさらに広げる。
    """
    base = 2.5
    # ボラティリティ調整
    if atr > atr_mean * 1.3:
        base += 0.5
    elif atr < atr_mean * 0.7:
        base -= 0.3

```

```

# RSI極値調整
if rsi < 30 or rsi > 70:
    base += 0.5
return max(base, 1.5)    # 最低でも1.5倍は確保

# -----
# メイン分析
# -----

def analyze_and_send() -> None:
    is_safe, reason = is_market_safe()
    if not is_safe:
        print(f"スキップ: {reason}")
        return

# --- 1. データ取得 -----
gold_1h = yf.download("GC=F", interval="60m", period="7d", progress=False)
gold_15m = yf.download("GC=F", interval="15m", period="5d", progress=False)
gold_d = yf.download("GC=F", period="2y", progress=False)
tnx = yf.download("^TNX", interval="60m", period="5d", progress=False)
dxy = yf.download("DX-Y.NYB", interval="60m", period="5d", progress=False)

if gold_1h.empty or gold_15m.empty or dxy.empty:
    print("データ取得失敗: スキップ")
    return

# MultiIndex 解除
for df in [gold_1h, gold_15m, gold_d, tnx, dxy]:
    if isinstance(df.columns, pd.MultiIndex):
        df.columns = df.columns.get_level_values(0)

# NaN行を除去（インデックス不一致によるNaN混入対策）
gold_1h.dropna(subset=['High', 'Low', 'Close'], inplace=True)
gold_15m.dropna(subset=['High', 'Low', 'Close'], inplace=True)
gold_d.dropna(subset=['Close'], inplace=True)
tnx.dropna(subset=['Close'], inplace=True)
dxy.dropna(subset=['Close'], inplace=True)

# --- 2. 指標計算 -----
try:
    atr = calc_atr(gold_1h)
    atr_mean = float(
        pd.concat([
            gold_1h['High'] - gold_1h['Low'],
            (gold_1h['High'] - gold_1h['Close'].shift()).abs(),
            (gold_1h['Low'] - gold_1h['Close'].shift()).abs(),
        ], axis=1).max(axis=1).rolling(14).mean().mean()
    )

```

```

    )
    adx_now, plus_di_last, minus_di_last = calc_adx(gold_15m)
    rsi_15 = calc_rsi(gold_15m['Close'])
except ValueError as e:
    print(f"指標計算エラー: {e}")
    return

now_p = float(gold_15m['Close'].iloc[-1])
jst_now = datetime.now(pytz.timezone('Asia/Tokyo'))
h = jst_now.hour

# — 3. ロジック判定 —————

trend_score = macro_score = logic_score = 0
messages: list[str] = []

# 長期トレンド (SMA200)
sma200 = float(gold_d['Close'].rolling(200).mean().iloc[-1])
is_uptrend = now_p > sma200
if is_uptrend:
    trend_score += 2
    messages.append("🟢長期上昇トレンド")
else:
    trend_score -= 2
    messages.append("🔴長期下落トレンド")

# 押し目・戻り売り (RSI)
if is_uptrend and rsi_15 <= 35:
    logic_score += 3
    messages.append(f"🔥絶好の押し目 (RSI:{rsi_15:.1f})")
elif not is_uptrend and rsi_15 >= 65:
    logic_score -= 3
    messages.append(f"❄️戻り売りの好機 (RSI:{rsi_15:.1f})")

# マクロ環境 (金利・ドル)
if len(tnx) >= 2 and len(dxy) >= 2:
    t_diff = float(tnx['Close'].iloc[-1]) - float(tnx['Close'].iloc[-2])
    dxy_diff = float(dxy['Close'].iloc[-1]) - float(dxy['Close'].iloc[-2])
    if t_diff < 0 and dxy_diff < 0:
        macro_score += 2
        messages.append("🌐マクロ追い風 (金利↓ドル↓)")
    elif t_diff > 0 and dxy_diff > 0:
        macro_score -= 2
        messages.append("🚫マクロ逆風 (金利↑ドル↑)")
    else:
        messages.append("⚠️マクロデータ不足")

# ADXトレンド加速

```

```

if adx_now > 25:
    if plus_di_last > minus_di_last:
        logic_score += 1
        messages.append(f"📈トレンド加速 (ADX:{adx_now:.1f})")
    else:
        logic_score -= 1
        messages.append(f"📉下落加速 (ADX:{adx_now:.1f})")

# フェイクブレイク判定
ph = float(gold_15m['High'].rolling(20).max().iloc[-2])
pl = float(gold_15m['Low'].rolling(20).min().iloc[-2])
last_bar = gold_15m.iloc[-1]
body      = abs(float(last_bar['Close']) - float(last_bar['Open']))
wick_up    = float(last_bar['High']) - max(float(last_bar['Open']), float(last_bar['Low']))

if float(last_bar['High']) > ph and (float(last_bar['High']) - ph) < atr * 0.5:
    if wick_up > body * 1.5:
        logic_score -= 1
        messages.append("⚠️フェイク上抜け（上ヒゲ陰転）")

if float(last_bar['Low']) < pl and (pl - float(last_bar['Low'])) < atr * 0.5:
    logic_score += 1
    messages.append("⚠️フェイク下抜け（下ヒゲ反転期待）")

# 時間帯スコア
time_score = 1 if 16 <= h <= 23 else (-1 if 0 <= h <= 8 else 0)

# — 4. 最終判定 —————
total_score = trend_score + macro_score + logic_score + time_score
abs_score    = abs(total_score)

if abs_score >= 3:
    direction = "BUY" if total_score > 0 else "SELL"
    sl_mult   = dynamic_sl_multiplier(rsi_15, atr, atr_mean)
    tp_mult   = sl_mult * 1.5 # RR比 1.5 固定 (sl_multに連動させて相対的に一定に)

    sl = now_p - (atr * sl_mult) if direction == "BUY" else now_p + (atr * sl_mult)
    tp = now_p + (atr * tp_mult) if direction == "BUY" else now_p - (atr * tp_mult)
    rr = tp_mult / sl_mult

    status = "🏆【極・推奨】" if abs_score >= 6 else "🎲【チャンス】"

lines = [
    f"{status} {direction}",
    f"信頼スコア: {abs_score}",
    "",
    *[f"・{m}" for m in messages],

```

```

        """
        f"💰 現在価格: ${now_p:.2f}",
        f"🎯 利確 (TP): ${tp:.2f}",
        f"🛡️ 損切 (SL): ${sl:.2f}",
        f"📊 RR比: {rr:.2f}",
        """
        f"🕒 RSI: {rsi_15:.1f} | ADX: {adx_now:.1f} | ATR: {atr:.2f}",
    ]
    send_line("\n".join(lines))
else:
    print(f"シグナルなし (スコア: {total_score}) ")

# -----
if __name__ == "__main__":
    analyze_and_send()

```